Machine Learning

Study Notes – Unit IV & Unit V

Dimensionality Reduction · Reinforcement Learning · Ensembles · Neural Networks

Contents at a glance

Unit IV Dimensionality Reduction (PCA, SVD) → Reinforcement Learning → Recurrent Reinforcement Learning → Bagging, Boosting, XGBoost

Unit V Neural Network Fundamentals → Activation Functions → Multilayer Feedforward Networks → CNN → RNN

Contents

UNIT IV **4**

1 Dimensionality Reduction **4**

1.1 What and Why . . . 4
1.2 Two Approaches . . . 4
1.3 Principal Component Analysis (PCA) . . . 5
1.3.1 Concept . . . 5
1.3.2 Steps of PCA . . . 5
1.3.3 Key Terms . . . 5
1.3.4 Properties and Limitations . . . 6
1.4 Singular Value Decomposition (SVD) . . . 6
1.4.1 Concept . . . 6
1.4.2 Truncated (Low-rank) SVD – used for dimensionality reduction . . . 6
1.4.3 SVD Procedure (steps) . . . 6
1.4.4 Applications . . . 7
1.4.5 PCA vs SVD . . . 7

2 Reinforcement Learning (RL) **7**

2.1 Concept . . . 7
2.2 The RL Loop . . . 7
2.3 Key Terms . . . 7
2.4 Exploration vs Exploitation . . . 8
2.5 Markov Decision Process (MDP) . . . 8
2.6 Basic Algorithm – Q-Learning . . . 8
2.7 Types of Learning Compared . . . 8
2.8 Applications . . . 9
2.9 Recurrent Reinforcement Learning (RRL) . . . 9
2.9.1 Concept . . . 9
2.9.2 Model . . . 9

2.9.3 Training . . . 9

3 Ensemble Learning: Bagging, Boosting, XGBoost 10
3.1 What is Ensemble Learning? . . . 10
3.2 Bagging (Bootstrap Aggregating) . . . 10
3.2.1 Concept . . . 10
3.2.2 Steps . . . 10
3.2.3 Random Forest – the best-known bagging method . . . 10
3.3 Boosting . . . 10
3.3.1 Concept . . . 10
3.3.2 AdaBoost (Adaptive Boosting) – basic idea . . . 11
3.3.3 Gradient Boosting – basic idea . . . 11
3.4 Bagging vs Boosting . . . 11
3.5 XGBoost (Extreme Gradient Boosting) . . . 11
3.5.1 Concept . . . 11
3.5.2 Objective Function . . . 11
3.5.3 Why XGBoost is Special . . . 12
3.5.4 Important Hyper-parameters . . . 12

UNIT V 13

4 Artificial Neural Networks: Fundamentals 13
4.1 What is a Neural Network? . . . 13
4.2 The Artificial Neuron (Perceptron) . . . 13
4.3 Layers of a Network . . . 13
4.4 How a Network Learns . . . 14
4.5 Loss Functions and Terms . . . 14

5 Activation Functions 14
5.1 Why Needed? . . . 14
5.2 Common Activation Functions . . . 14

6 Multilayer Feedforward Networks 15
6.1 Single-layer vs Multilayer . . . 15
6.2 Computation in an MLP . . . 15
6.3 Training an MLP – Backpropagation in Brief . . . 16
6.4 Gradient Descent Variants . . . 16
6.5 Design Choices . . . 16

7 Convolutional Neural Network (CNN) 16
7.1 Why CNN? . . . 16
7.2 Architecture . . . 16
7.3 Convolution Layer . . . 17
7.4 Activation (ReLU) in CNN . . . 17
7.5 Pooling Layer . . . 18

7.6 Flatten and Fully Connected Layers . . . 18
7.7 CNN Summary . . . 18

8 Recurrent Neural Network (RNN) 18
8.1 Why RNN? . . . 18
8.2 Structure . . . 19
8.3 Equations . . . 19
8.4 Types of RNN Architectures . . . 19
8.5 Training: Backpropagation Through Time (BPTT) . . . 19
8.6 Problems of Simple RNN . . . 19
8.7 Improved Variants (overview) . . . 20
8.8 Applications . . . 20
8.9 ANN vs CNN vs RNN . . . 20

9 Textbooks to Refer 21

UNIT IV

1 Dimensionality Reduction

1.1 What and Why

- **Dimensionality** = number of features (columns) in a dataset.
- **Dimensionality reduction** = converting data with many features into fewer features **while keeping most of the useful information**.
- Problem it solves: the **curse of dimensionality** – as features grow, data becomes sparse, distances lose meaning, models overfit and training becomes slow.

Benefit	Explanation
Less overfitting	Fewer features ⇒ simpler model
Faster training	Less computation and memory
Visualization	Reduce to 2D / 3D for plotting
Noise removal	Low-information directions are dropped
Removes multi-collinearity	Correlated features are merged into uncorrelated ones

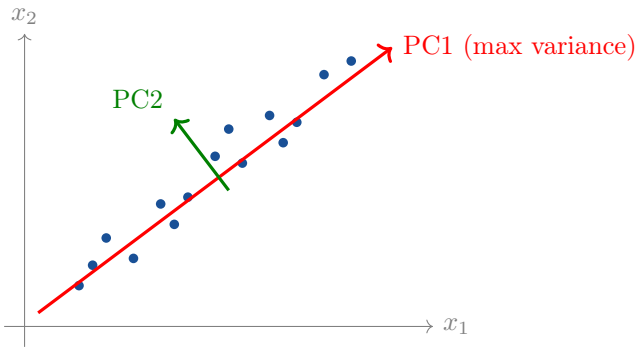
1.2 Two Approaches

	Feature Selection	Feature Extraction
Idea	Pick a *subset* of original features	Create *new* features from combinations of old ones
Original meaning	Preserved	Lost (new axes)
Examples	Correlation filter, forward/backward selection	**PCA, SVD**

1.3 Principal Component Analysis (PCA)

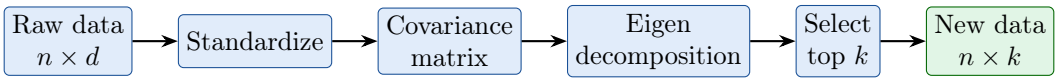
1.3.1 Concept

- PCA is an **unsupervised, linear** feature-extraction technique.
- It finds new perpendicular axes called **principal components (PCs)** along which the data has the **maximum variance**.
- PC1 = direction of largest variance, PC2 = next largest and **orthogonal** to PC1, and so on.
- Keep the first k PCs ($k < d$) and drop the rest.



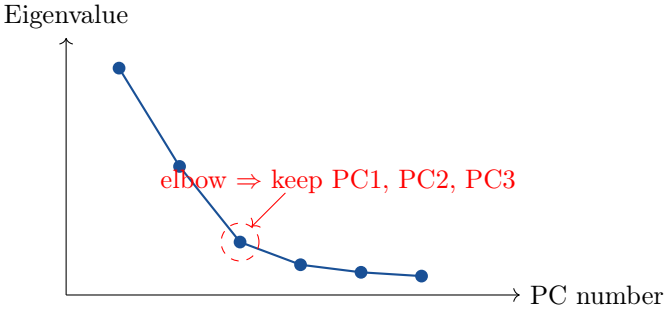
1.3.2 Steps of PCA

1. **Standardize** the data: $z = \dfrac{x - \mu}{\sigma}$ (mean 0, variance 1) so that no feature dominates due to scale.
2. Compute the **covariance matrix** $C = \dfrac{1}{n-1} Z^T Z$ (size $d \times d$).
3. Compute **eigenvalues** (λ) and **eigenvectors** (v) of C: $Cv = \lambda v$.
4. **Sort** eigenvectors by eigenvalue (largest first).
5. **Select top** k eigenvectors $\rightarrow$ projection matrix W ($d \times k$).
6. **Project**: $Z_{\text{new}} = Z\,W$ (size $n \times k$).



1.3.3 Key Terms

- **Eigenvector** = direction of a principal component. **Eigenvalue** = amount of variance along that direction.
- **Explained variance ratio** of $\text{PC}_i = \dfrac{\lambda_i}{\sum_j \lambda_j}$.
- **Choosing k:** keep enough PCs for a cumulative variance of about 90–95%, or use a **scree plot** (look for the "elbow").



Scree plot

1.3.4 Properties and Limitations

Advantages	Limitations
Reduces dimensions, speeds up models	PCs are hard to interpret
PCs are uncorrelated	Only captures *linear* relationships
Good for visualization and noise removal	Sensitive to scale (must standardize)
Unsupervised – needs no labels	Some information is always lost

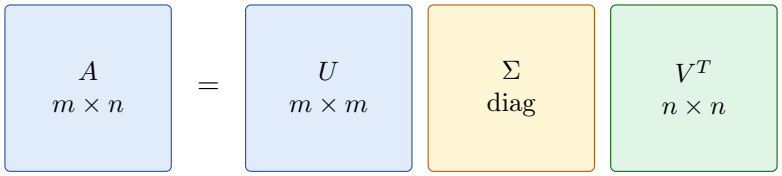
1.4 Singular Value Decomposition (SVD)

1.4.1 Concept

- SVD factorizes *any* matrix A $(m \times n)$ into three matrices:

$$\boxed{A = U\,\Sigma\,V^T}$$

Matrix	Size	Meaning
U	$m \times m$	Left singular vectors (orthogonal columns) – describe *rows* (samples)
Σ	$m \times n$	Diagonal; singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ – "importance" of each direction
V^T	$n \times n$	Right singular vectors (orthogonal) – describe *columns* (features)



1.4.2 Truncated (Low-rank) SVD – used for dimensionality reduction

- Keep only the top k singular values: $A \approx U_k \Sigma_k V_k^T$.
- Reduced data: $A_{\text{reduced}} = U_k \Sigma_k$ (size $m \times k$).
- Larger singular value $\Rightarrow$ more information; tiny ones are treated as noise and dropped.

1.4.3 SVD Procedure (steps)

1. Compute $A^T A$; its eigenvalues are σ_i^2 and eigenvectors give columns of V.
2. $\sigma_i = \sqrt{\lambda_i}$ (arranged in descending order) $\rightarrow \Sigma$.
3. Compute $u_i = \dfrac{1}{\sigma_i} A v_i \rightarrow U$.
4. Keep top k components for reduction.

1.4.4 Applications

- Dimensionality reduction and noise removal
- Image compression (store only top k values)
- Recommender systems (user–item matrix factorization)
- Text mining – Latent Semantic Analysis (LSA)

1.4.5 PCA vs SVD

	PCA	SVD
Works on	Covariance matrix (square, symmetric)	Any $m \times n$ matrix directly
Core tool	Eigen decomposition	Singular value decomposition
Centering	Data must be mean-centered	Not required in general
Relation	PCA can be computed using SVD of the centered data: $\sigma_i^2/(n-1) = \lambda_i$, and V = principal directions.	

2 Reinforcement Learning (RL)

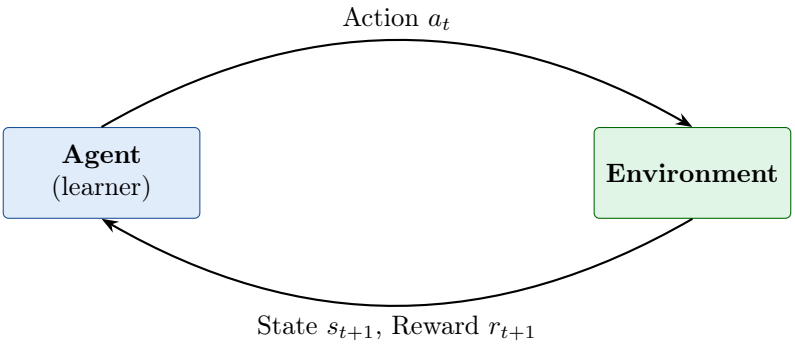
2.1 Concept

- RL is learning by **trial and error**: an **agent** interacts with an **environment**, takes **actions**, and receives **rewards** or penalties.
- Goal: learn a strategy (**policy**) that **maximizes the total cumulative reward** over time.
- No labelled data – feedback is a reward signal, often delayed.

Example

Training a dog: sit correctly → treat (reward); misbehave → no treat. The dog slowly learns which actions earn treats.

2.2 The RL Loop



2.3 Key Terms

Return (total reward)

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Term	Meaning
Agent	The learner / decision maker
Environment	Everything the agent interacts with
State (s)	Current situation of the environment
Action (a)	A move the agent can make
Reward (r)	Numerical feedback after an action
Policy (π)	Agent's strategy: mapping from state to action, $\pi(s) \to a$
Value function $V(s)$	Expected total future reward starting from state s
Q-value $Q(s, a)$	Expected total future reward after taking action a in state s
Discount factor (γ)	$0 \leq \gamma \leq 1$; how much future rewards matter ($\gamma \to 0$: short-sighted, $\gamma \to 1$: far-sighted)
Episode	One complete run from start state to terminal state

2.4 Exploration vs Exploitation

- **Exploration** – try new actions to discover better rewards.
- **Exploitation** – use the best action known so far.
- **ϵ-greedy**: with probability ϵ explore (random action), with probability $1 - \epsilon$ exploit (best action).

2.5 Markov Decision Process (MDP)

- RL problems are formalized as an MDP $= (S, A, P, R, \gamma)$: states, actions, transition probabilities, rewards, discount.
- **Markov property**: the next state depends only on the current state and action, not on the full history.

2.6 Basic Algorithm – Q-Learning

- Maintains a **Q-table** (rows = states, columns = actions) storing $Q(s, a)$.
- Update rule after each step:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big]$$

- α = learning rate; $r + \gamma \max Q(s', a')$ = target; bracket = error.
- Repeat for many episodes until Q-values stabilize; then policy = pick action with highest Q in each state.

2.7 Types of Learning Compared

	Supervised	Unsupervised	Reinforcement
Data	Labelled	Unlabelled	No fixed data; interaction
Feedback	Correct answer given	None	Reward / penalty
Goal	Predict output	Find structure	Maximize reward
Example	Spam detection	Clustering	Game playing, robots

2.8 Applications

Game playing (chess, Go), robotics, self-driving cars, recommendation systems, algorithmic trading, resource scheduling.

2.9 Recurrent Reinforcement Learning (RRL)

2.9.1 Concept

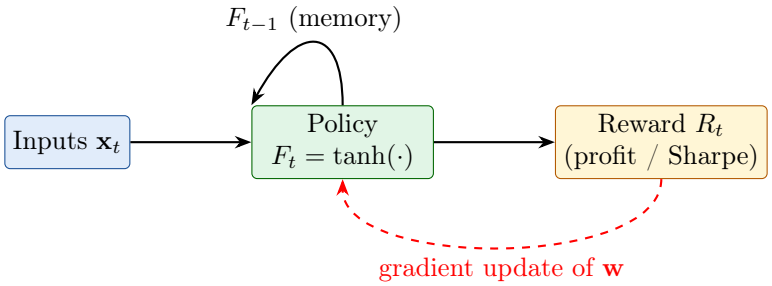
- RRL combines a **recurrent neural network** with reinforcement learning.
- The policy **remembers its previous action/output** through a feedback (recurrent) connection, so decisions depend on the **past**.
- It is a **direct** RL method: it learns the policy directly by optimizing a performance measure, *without* building a value table / Q-function.
- Widely used in **financial trading** (buy / sell / hold) – transaction costs depend on the previous position, so memory is essential.

2.9.2 Model

- Output (action/position) at time t:

$$F_t = \tanh\big(\mathbf{w}^T \mathbf{x}_t + u\, F_{t-1} + b\big)$$

- $\mathbf{x}_t$ = current inputs (e.g. recent returns); F_{t-1} = previous output (the recurrent feedback); $\mathbf{w}, u, b$ = learnable parameters.
- $F_t \in [-1, 1]$: -1 = short/sell, 0 = neutral, $+1$ = long/buy.



2.9.3 Training

1. Compute actions $F_1, \ldots, F_T$ using the recurrent policy.
2. Compute reward/utility U (profit or **Sharpe ratio** = mean return / std of return).
3. Update parameters by **gradient ascent**: $\mathbf{w} \leftarrow \mathbf{w} + \rho \dfrac{\partial U}{\partial \mathbf{w}}$ (gradient flows back through time).
4. Repeat until the utility stops improving.

	Q-Learning (value based)	**RRL (direct policy)**
Learns	Value table $Q(s, a)$	Policy directly
Actions	Discrete	Continuous or discrete
Memory	Only current state	Recurrent feedback of past output
Objective	Expected cumulative reward	Any utility (profit, Sharpe ratio)

3 Ensemble Learning: Bagging, Boosting, XGBoost

3.1 What is Ensemble Learning?

- Combine **many models (weak learners)** to get one stronger model: "wisdom of the crowd".
- Final output: **voting** (classification) or **averaging** (regression).
- Why it works: individual errors cancel out ⇒ lower variance and/or bias.

Error type	Meaning	Reduced by
High variance (overfit)	Model too sensitive to training data	**Bagging**
High bias (underfit)	Model too simple	**Boosting**

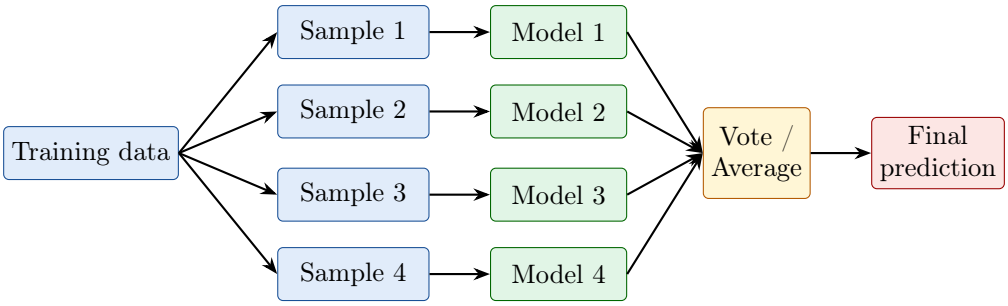
3.2 Bagging (Bootstrap Aggregating)

3.2.1 Concept

- Train models **independently, in parallel** on different random samples; then combine.
- **Bootstrap sample** = random sample of size n drawn *with replacement* (about 63% unique rows; remaining ≈37% = **out-of-bag (OOB)** data, usable for validation).
- Aggregation: majority vote (classification), average (regression).

3.2.2 Steps

1. Create B bootstrap samples from the training data.
2. Train one base model (usually a decision tree) on each sample.
3. Combine predictions by vote / average.



3.2.3 Random Forest – the best-known bagging method

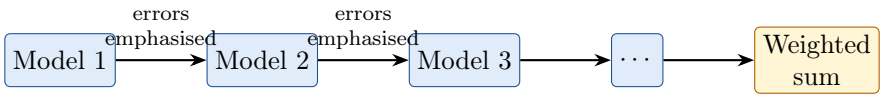
- Bagging of decision trees + **random subset of features at each split** ⇒ trees become less correlated.
- Works for both classification and regression.

3.3 Boosting

3.3.1 Concept

- Train models **sequentially**; each new model **focuses on the mistakes** of previous ones.
- Weak learners (e.g. shallow trees / "stumps") are combined into a strong learner as a **weighted sum**.

- Mainly reduces **bias**; can overfit if run for too long / noisy data.



3.3.2 AdaBoost (Adaptive Boosting) – basic idea

1. Give every sample equal weight $w_i = 1/n$.
2. Train a weak learner; compute its error ε.
3. Learner's importance: $\alpha = \frac{1}{2}\ln\dfrac{1-\varepsilon}{\varepsilon}$ (lower error $\Rightarrow$ larger α).
4. **Increase weights** of misclassified samples and decrease the others; normalize.
5. Repeat for M rounds. Final: $H(x) = \text{sign}\left(\sum_m \alpha_m h_m(x)\right)$.

3.3.3 Gradient Boosting – basic idea

- Start with a simple prediction (e.g. mean of y).
- Each new tree is trained to predict the **residuals** (errors) of the current model.
- Update: $F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$, where η = **learning rate** (shrinkage).
- Residual = negative gradient of the loss, hence the name *gradient* boosting.

3.4 Bagging vs Boosting

Aspect	Bagging	Boosting
Training	Parallel, independent	Sequential, dependent
Data for each model	Random bootstrap sample	Re-weighted data / residuals
Main goal	Reduce **variance**	Reduce **bias**
Combination	Equal vote / average	Weighted sum
Base learners	Strong, complex (deep trees)	Weak, simple (shallow trees)
Overfitting risk	Low	Higher
Examples	Random Forest	AdaBoost, Gradient Boosting, XGBoost

3.5 XGBoost (Extreme Gradient Boosting)

3.5.1 Concept

- An **optimized, scalable implementation of gradient boosted decision trees**; very popular in competitions and industry.
- Builds trees sequentially; each tree corrects the errors of the earlier ones.

3.5.2 Objective Function

$$\text{Obj} = \underbrace{\sum_i l(y_i, \hat{y}_i)}_{\text{training loss}} + \underbrace{\sum_k \Omega(f_k)}_{\text{regularization}}, \qquad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \sum_j w_j^2$$

- T = number of leaves, w_j = leaf weights; γ, λ control complexity $\Rightarrow$ **prevents overfitting**.

3.5.3 Why XGBoost is Special

Feature	Benefit
Regularization (L1 & L2)	Controls model complexity, less overfitting
Handles missing values	Learns the best default direction at each split
Parallel processing	Fast tree construction
Tree pruning (max depth, γ)	Removes splits that give little gain
Shrinkage (learning rate η)	Each tree contributes only a small step
Column & row subsampling	Reduces overfitting and speeds up training
Built-in cross-validation, early stopping	Stops when validation score stops improving

3.5.4 Important Hyper-parameters

Parameter	Meaning
`n_estimators`	Number of trees
`learning_rate` (η)	Step size; smaller = slower but safer
`max_depth`	Maximum depth of each tree
`subsample`	Fraction of rows used per tree
`colsample_bytree`	Fraction of features used per tree
`gamma`, `lambda`, `alpha`	Minimum split gain, L2 and L1 regularization

Remember

Bagging = many strong models, parallel, vote → less variance. **Boosting** = many weak models, sequential, fix errors → less bias. **XGBoost** = gradient boosting + regularization + speed.

UNIT V

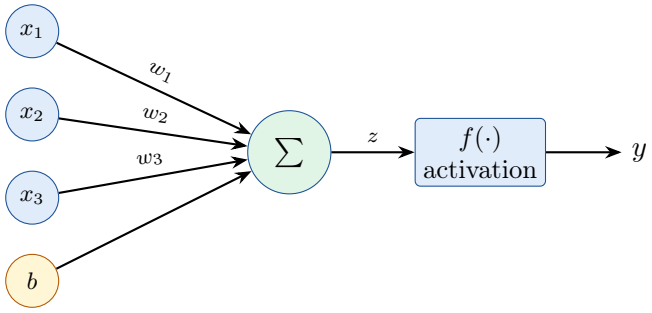
4 Artificial Neural Networks: Fundamentals

4.1 What is a Neural Network?

- An **Artificial Neural Network (ANN)** is a model inspired by the human brain, made of many connected simple units (**neurons**) arranged in layers.
- It learns patterns from data by adjusting the **weights** of the connections.
- Basis of **deep learning** (networks with many hidden layers).

Biological neuron	Artificial neuron
Dendrites (receive signals)	Inputs x_i
Synapse strength	Weights w_i
Cell body (sums signals)	Weighted sum + bias
Firing threshold	Activation function
Axon (sends output)	Output y

4.2 The Artificial Neuron (Perceptron)



$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b, \qquad y = f(z)$$

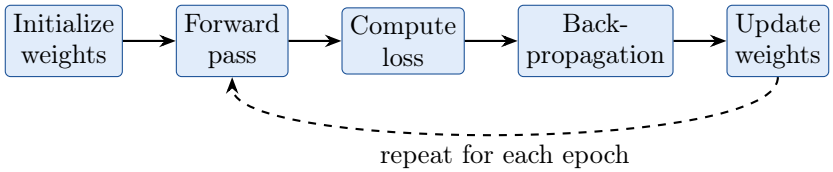
- **Weights** w_i: importance of each input (learned).
- **Bias** b: shifts the activation threshold, lets the neuron fire even if all inputs are 0.
- **Activation function** f: adds non-linearity.

4.3 Layers of a Network

- **Input layer**: receives raw features (no computation; one node per feature).
- **Hidden layer(s)**: perform computation and extract features.
- **Output layer**: gives final prediction (1 node for regression/binary, K nodes for K classes).

4.4 How a Network Learns

1. **Initialize** weights randomly (small values).
2. **Forward propagation**: inputs pass layer by layer to produce a prediction $\hat{y}$.
3. **Loss**: compare $\hat{y}$ with true y.
4. **Backpropagation**: compute gradient of the loss w.r.t. every weight using the chain rule, from output back to input.
5. **Gradient descent update**: $w \leftarrow w - \eta \dfrac{\partial L}{\partial w}$ (η = learning rate).
6. Repeat over many **epochs**.



4.5 Loss Functions and Terms

Task	Loss	Output activation
Regression	Mean Squared Error $\frac{1}{n}\sum(y-\hat{y})^2$	Linear
Binary classification	Binary cross-entropy	Sigmoid
Multi-class classification	Categorical cross-entropy	Softmax

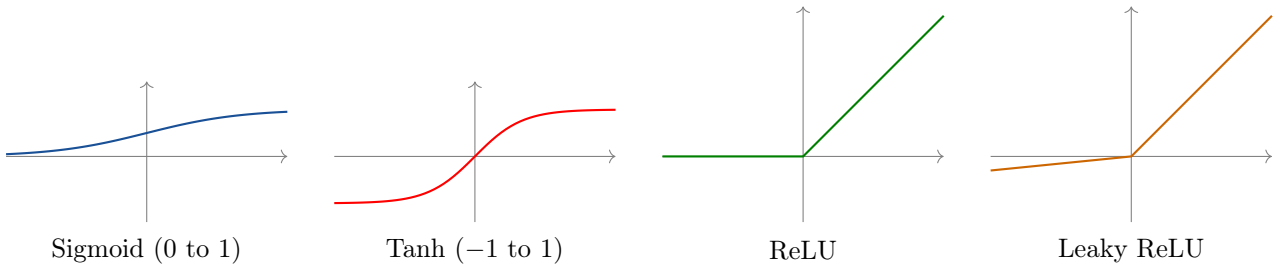
- **Epoch**: one full pass over the training data. **Batch size**: samples used per weight update. **Learning rate**: step size of updates.
- **Overfitting** remedies: more data, early stopping, **dropout**, regularization.

5 Activation Functions

5.1 Why Needed?

- Without activation functions a network is just a **linear** model, however many layers it has.
- Activations introduce **non-linearity**, so the network can learn complex patterns (e.g. XOR).
- Desirable: non-linear, differentiable (for backpropagation), cheap to compute.

5.2 Common Activation Functions



Quick choice guide

Hidden layers → **ReLU**. Binary output → **Sigmoid**. Multi-class output → **Softmax**. Regression output → **Linear**. Sequence models (RNN) → **Tanh**.

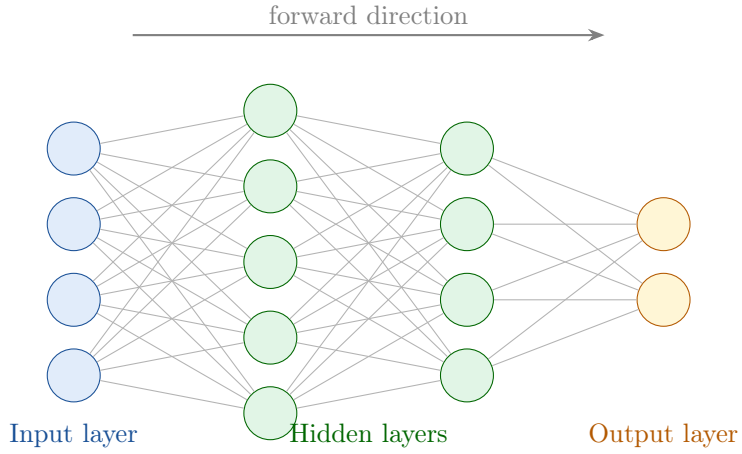
Function	Formula	Range	Used in	Drawback
Step (threshold)	$f = 1$ if $z \geq 0$, else 0	$\{0, 1\}$	Original perceptron	Not differentiable; no gradients
Linear	$f(z) = z$	$(-\infty, \infty)$	Regression output	No non-linearity
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Binary output layer	Vanishing gradient; not zero-centred
Tanh	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$(-1, 1)$	Hidden layers, RNN	Vanishing gradient
ReLU	$\max(0, z)$	$[0, \infty)$	**Default for hidden layers**	"Dying ReLU" (neurons stuck at 0)
Leaky ReLU	z if $z > 0$, else $0.01z$	$(-\infty, \infty)$	Fix for dying ReLU	Slope is a choice
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0, 1)$, sums to 1	Multi-class output	Only for output layer

Vanishing gradient: for sigmoid/tanh, gradients become extremely small in deep networks, so early layers learn very slowly. ReLU largely avoids this.

6 Multilayer Feedforward Networks

6.1 Single-layer vs Multilayer

- **Single-layer perceptron**: only input and output layers; can solve only **linearly separable** problems (AND, OR) – fails on **XOR**.
- **Multilayer Perceptron (MLP) / Feedforward network**: one or more *hidden layers*; can learn non-linear decision boundaries.
- **Feedforward**: information flows in **one direction** only – input → hidden → output, no loops.
- **Fully connected (dense)**: every neuron connects to all neurons of the next layer.



6.2 Computation in an MLP

For layer l with weight matrix $W^{[l]}$, bias $b^{[l]}$ and activation f:

$$a^{[l]} = f\big(W^{[l]}a^{[l-1]} + b^{[l]}\big), \qquad a^{[0]} = x, \quad a^{[L]} = \hat{y}$$

6.3 Training an MLP – Backpropagation in Brief

1. Forward pass gives $\hat{y}$; compute loss L.
2. Output-layer error: $\delta^{[L]} = \hat{y} - y$ (for softmax/sigmoid + cross-entropy).
3. Propagate error backwards: $\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \odot f'(z^{[l]})$.
4. Gradients: $\dfrac{\partial L}{\partial W^{[l]}} = \delta^{[l]} \, (a^{[l-1]})^T$; update using gradient descent.

6.4 Gradient Descent Variants

Variant	Samples per update	Property
Batch GD	Entire dataset	Stable but slow
Stochastic GD (SGD)	1 sample	Fast, noisy
Mini-batch GD	Small group (e.g. 32, 64)	Best trade-off; most used

6.5 Design Choices

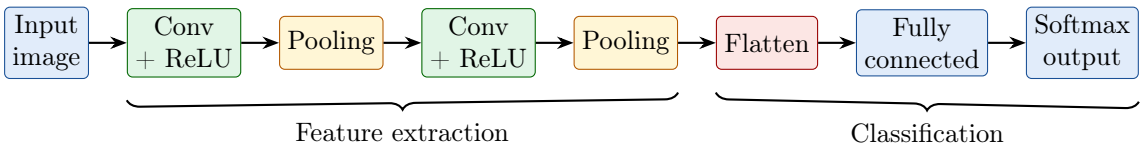
- Number of hidden layers and neurons, activation functions, loss function, learning rate, batch size, epochs.
- **Universal approximation**: an MLP with one hidden layer and enough neurons can approximate any continuous function.

7 Convolutional Neural Network (CNN)

7.1 Why CNN?

- Designed for **grid-like data – mainly images**.
- A normal MLP on a 100×100 image needs 10,000 inputs per neuron ⇒ too many parameters, ignores spatial structure.
- CNN uses **local connections** and **shared weights** ⇒ few parameters, detects patterns anywhere in the image.

7.2 Architecture



7.3 Convolution Layer

- A small matrix called a **filter** / **kernel** (e.g. 3×3) **slides** over the image; at each position multiply element-wise and sum → one value in the **feature map**.
- Different filters detect different features: edges → textures → shapes → objects (deeper layers learn more complex features).
- Filter weights are **learned** by backpropagation; the same filter is reused over the entire image (**weight sharing**).

Convolution example: 3×3 input, 2×2 filter

1	2	0
0	1	3
2	1	1

Input

$*$

1	0
0	1

Filter

$=$

2	5
1	2

Feature map

Top-left value: $1 \cdot 1 + 2 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 = 2$. Slide the filter right/down by the stride for the other values.

Key hyper-parameters

Parameter	Meaning
Filter size	Usually 3×3 or 5×5
Number of filters	Number of feature maps produced (depth of output)
Stride (S)	Step size of the sliding window; larger stride ⇒ smaller output
Padding (P)	Zeros added around the border; "same" keeps the size, "valid" = no padding

Output size

$$\text{Output} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

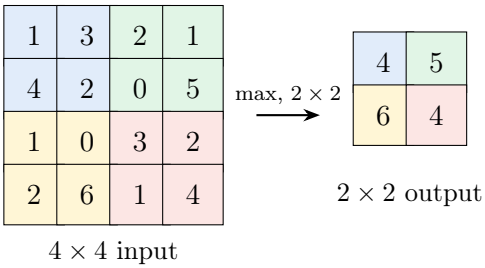
N = input size, F = filter size, P = padding, S = stride. Example: $N = 5, F = 3, P = 0, S = 1 \Rightarrow 3$.

7.4 Activation (ReLU) in CNN

Applied after convolution to every value of the feature map: negatives become 0, positives stay. Adds non-linearity.

7.5 Pooling Layer

- **Down-samples** each feature map: reduces size, computation and overfitting; gives small shift-invariance.
- Has **no learnable parameters**.
- **Max pooling**: take the maximum of each window (most common). **Average pooling**: take the average.



7.6 Flatten and Fully Connected Layers

- **Flatten**: convert the final 2D/3D feature maps into a 1D vector.
- **Fully connected layers**: combine the extracted features to make the decision.
- **Output**: Softmax (multi-class) or Sigmoid (binary).

7.7 CNN Summary

Layer	Purpose	Learnable?
Convolution	Extract features (edges, shapes)	Yes (filters)
ReLU	Non-linearity	No
Pooling	Reduce size, keep key info	No
Flatten	Convert to vector	No
Fully connected	Classification	Yes (weights)

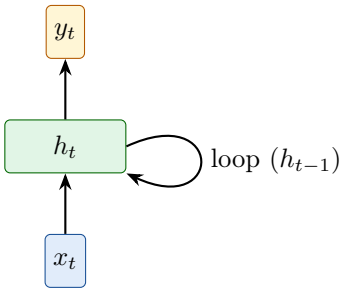
Applications: image classification, object detection, face recognition, medical imaging, self-driving cars.
Well-known CNNs: LeNet, AlexNet, VGG, ResNet.

8 Recurrent Neural Network (RNN)

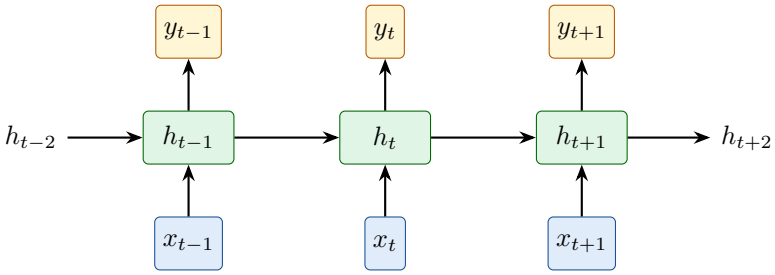
8.1 Why RNN?

- Designed for **sequential data** where **order matters**: text, speech, time series, stock prices.
- Feedforward networks/CNNs treat inputs independently and have **no memory**.
- RNN has a **hidden state** h_t – a memory of what it has seen so far – and **shares the same weights at every time step**.
- Inputs and outputs can be of **variable length**.

8.2 Structure



Folded form



Unfolded through time (same weights W every step)

8.3 Equations

$$h_t = \tanh\big(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b_h\big), \qquad y_t = g\big(W_{hy}\, h_t + b_y\big)$$

- W_{xh}: input → hidden; W_{hh}: hidden → hidden (memory); W_{hy}: hidden → output; $h_0 = 0$.
- g = Softmax / Sigmoid / Linear depending on the task.

8.4 Types of RNN Architectures

Type	Input → Output	Example
One-to-One	1 → 1	Ordinary neural network
One-to-Many	1 → sequence	Image captioning, music generation
Many-to-One	sequence → 1	Sentiment analysis, spam detection
Many-to-Many (equal length)	sequence → sequence	Named entity tagging, video frame labelling
Many-to-Many (different length)	sequence → sequence	Machine translation (encoder–decoder)

8.5 Training: Backpropagation Through Time (BPTT)

- Unfold the network over all time steps, compute the loss at each step, and backpropagate **through time**.
- Gradients of the shared weights are **summed** over all time steps, then weights are updated by gradient descent.

8.6 Problems of Simple RNN

Problem	Explanation
Vanishing gradient	Gradients shrink over many steps ⇒ cannot learn long-term dependencies
Exploding gradient	Gradients grow very large ⇒ unstable training (fixed by **gradient clipping**)
Slow training	Steps are sequential, cannot be parallelized

8.7 Improved Variants (overview)

Variant	Idea
LSTM (Long Short-Term Memory)	Adds a **cell state** and three gates: **forget** (what to erase), **input** (what to store), **output** (what to reveal). Remembers long-term information
GRU (Gated Recurrent Unit)	Simplified LSTM with two gates: **update** and **reset**; fewer parameters, faster
Bidirectional RNN	Reads the sequence forward and backward; uses past and future context

8.8 Applications

Language modelling and text generation, machine translation, speech recognition, sentiment analysis, time-series and stock forecasting.

8.9 ANN vs CNN vs RNN

	ANN (MLP)	CNN	RNN
Best for	Tabular data	Images / grids	Sequences
Connections	Fully connected	Local + shared filters	Recurrent loop
Memory	None	None	Hidden state
Input size	Fixed	Fixed (usually)	Variable length
Key layer	Dense	Convolution, pooling	Recurrent cell
Example	Credit scoring	Face recognition	Translation

Final Quick Revision

- **PCA**: standardize → covariance → eigenvectors → top k → project. **SVD**: $A = U\Sigma V^T$, keep top k singular values.
- **RL**: agent–environment–reward loop; goal = maximize cumulative reward; Q-learning updates a Q-table. **RRL**: recurrent policy F_t trained directly on profit/Sharpe ratio.
- **Bagging**: parallel, bootstrap, reduces variance (Random Forest). **Boosting**: sequential, fixes errors, reduces bias (AdaBoost, GBM). **XGBoost**: regularized, fast gradient boosting.
- **Neuron**: $y = f(\mathbf{w}^T\mathbf{x} + b)$. Learning = forward pass → loss → backprop → gradient descent.
- **Activations**: ReLU (hidden), Sigmoid (binary), Softmax (multi-class), Tanh (RNN).
- **CNN**: Convolution → ReLU → Pooling → Flatten → Dense. **RNN**: hidden state with memory, BPTT; LSTM/GRU fix vanishing gradients.

9 Textbooks to Refer

- Machine Learning: Concepts, Techniques and Applications By T V Geetha, S Sendhilkumar
- Machine Learning Subramanian Chandramouli, Saikat Dutt, Amita Dāśa